

Discussion Unit: Why DSE Analytics

(Introduction to Analytics)

- Understand the Why of Analytics
- Four Primary Functional Areas to DSE
- History of Apache Spark (history of Apache Hadoop)
- 5 Functional Areas to (DSE Analytics)
- Business Value of DSE Analytics, Use Cases

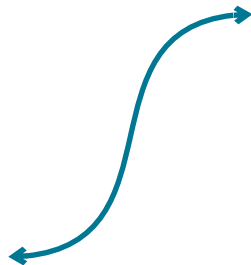
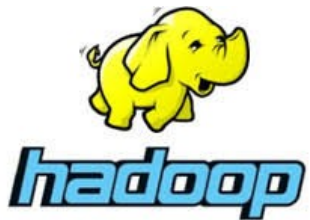


Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

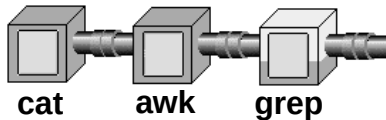
DSE Analytics; Hadoop versus Spark, languages and functions

DSE Analytics: Hadoop versus Spark

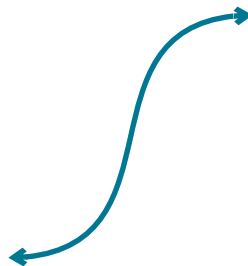


- ☐ Distributed Filesystem (HDFS)
- ☐ Computing engine, not a storage system
- ☐ Composable API based on functional programming
- ☐ Operates on Key/Value (K/V) pairs
- ☐ K/V classes have to be serializable by the framework
- ☐ M/R: Input reader, Mapper, Partitioner, Comparator, Reducer, Output Writer
- ☐ RDD, DataFrame/Dataset
- ☐ Distributed programming model, user specifies transforms to build up a directed acyclic graph

```
(function() {  
  return "Hello World";  
})();
```



Language Adjectives/Terms:



☐ Declarative

☐ Structured Programming

☐ Fluent API

☐ Lazy Evaluation

☐ Lambda

☐ Imperative Programming

☐ Object Oriented

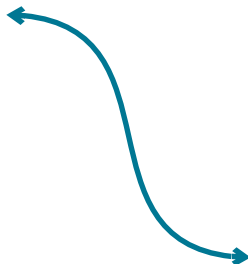
☐ Functional Programming

In Many Languages ..

{ }



()



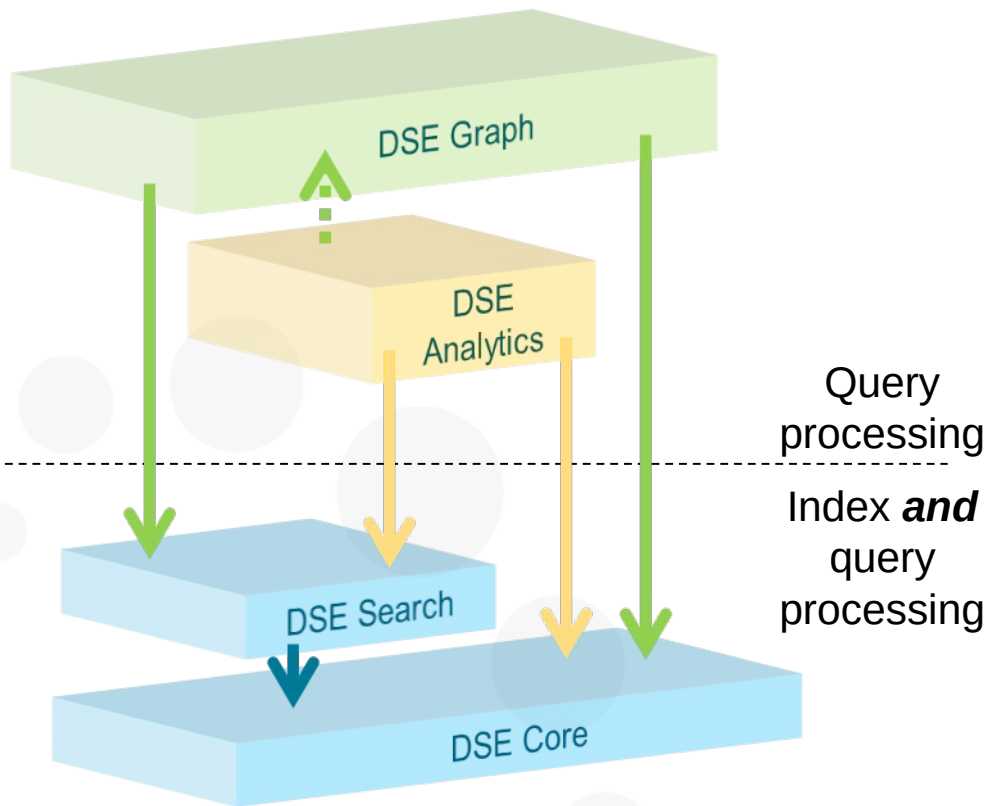
Function Body (or Block), if not named, it is an anonymous function, or lambda



Parameters to Function

End of Discussion Lab:

The 4 Primary Functional Areas to DSE



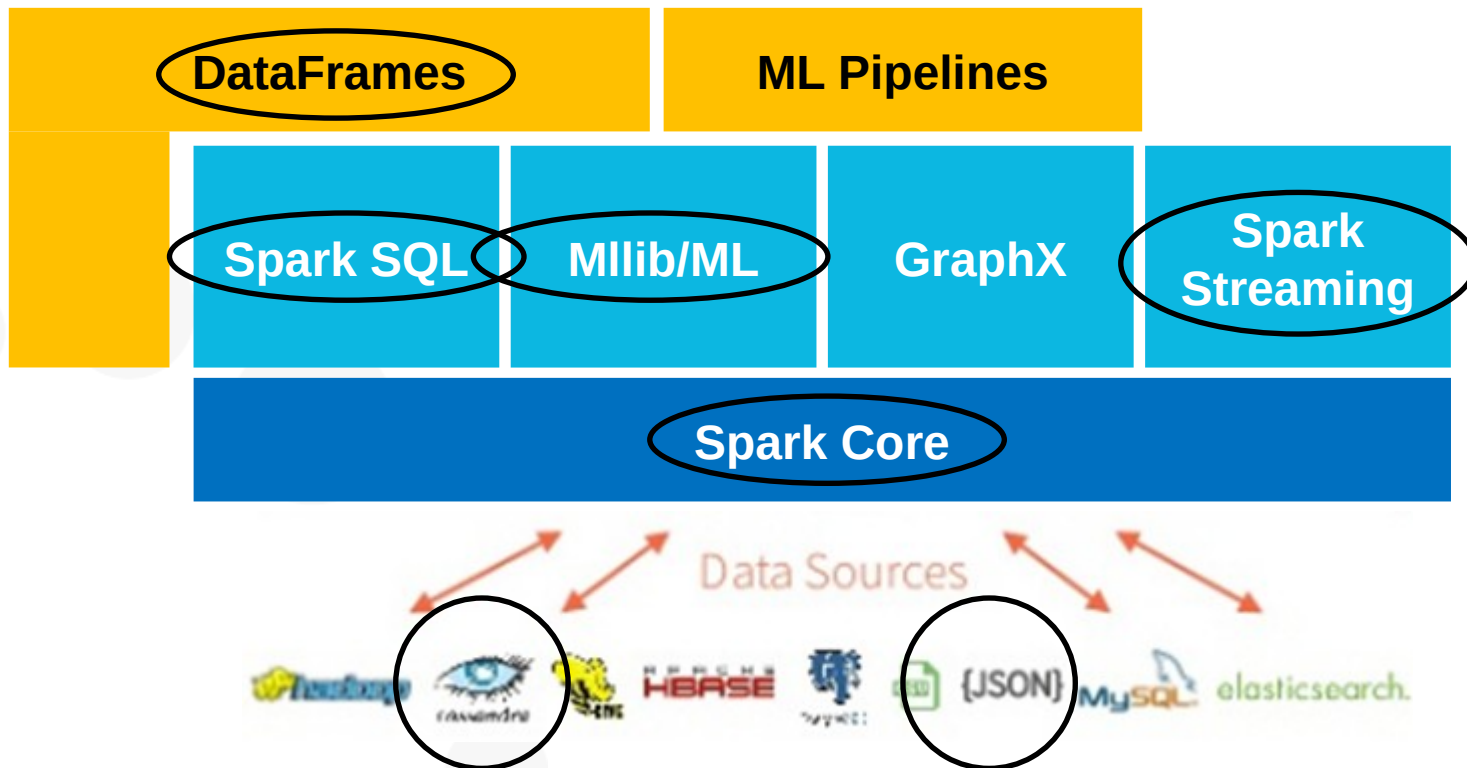
- All 4 primary functional areas provide query processing

The Why



- DSE Analytics
 - *Parallel Query Processing*
 - Horizontal scaling
 - High Speed
 - Batch, Streaming, Iterative, Interactive

DSE Analytics: 5 Major Functional Areas



DSE Analytics: What is it-

Enhanced version of Apache Spark

- DSE manages Spark Master and auto restarts without any other third party software
 - Restart of a Master with information on who is Master to whole cluster
 - Restart of any failed workers
- Ability to submit jobs to any Spark nodes, rather than having to know who the master is
- With DSE to Cassandra the data sits on the nodes with Spark, **less data shuffling**
 - Spark attempts to optimize jobs to pull data on local node primarily
 - In DSE 5.1 and above, both the Master and Worker run in the same JVM as DSE
 - **Able to inherit unified DSE security features**
- DSE drivers to Cassandra
 - Continuous paging for speed in returning data
 - **Integrate DSE Graph with Spark GraphFrames**
 - **Integration with DSE Search**
- Always on SQL Server replacing the non fault tolerant Thrift Server

DSE Analytics: What is it-

Integrated Platform for Advanced Analytics

- An end-to-end, integrated platform; Spark has no persistent data tier
 - One-stop-shop for data ingestion, transformation and analytics (SparkSQL, MLlib, ML) via Spark integration
- Workload isolation - No data movement between OLTP and OLAP systems
- Transactional data immediately available for analysis
- Drill down from analysis to current transactional data
- Reduce the need for multiple copies of same data

DSE Analytics: What is it-

Integrated Platform for Operational Analytics

- Integrated with DSE
 - Masterless, distributed architecture
 - Multi-Data Center support
 - Multi-model platform
- Inherits all the enterprise-class benefits of DSE:
 - Contextual insights (with Graph capability)
 - Always-on
 - Massively scalable
 - High-performance
 - **Ease to use and manage, one less cluster**

DSE Analytics: Key Use Cases

DataStax customers use DSE Analytics in the following areas:

- **Data Ingestion** from streaming sources, with/without transformation/aggregation
- Data Quality – primarily with respect to data ingestion
- ETL – from one internal store to another and from external sources to internal stores
- Pre-calculated Aggregates – scheduled “mini-batch”
- **Machine Learning on Operational Data**
- **Real-time personalization & recommendation**
- Business Intelligence – via ODBC/JDBC connectivity
- Primarily Tableau, working on partnering with additional BI vendors

Objections: Why use OSS Apache Spark

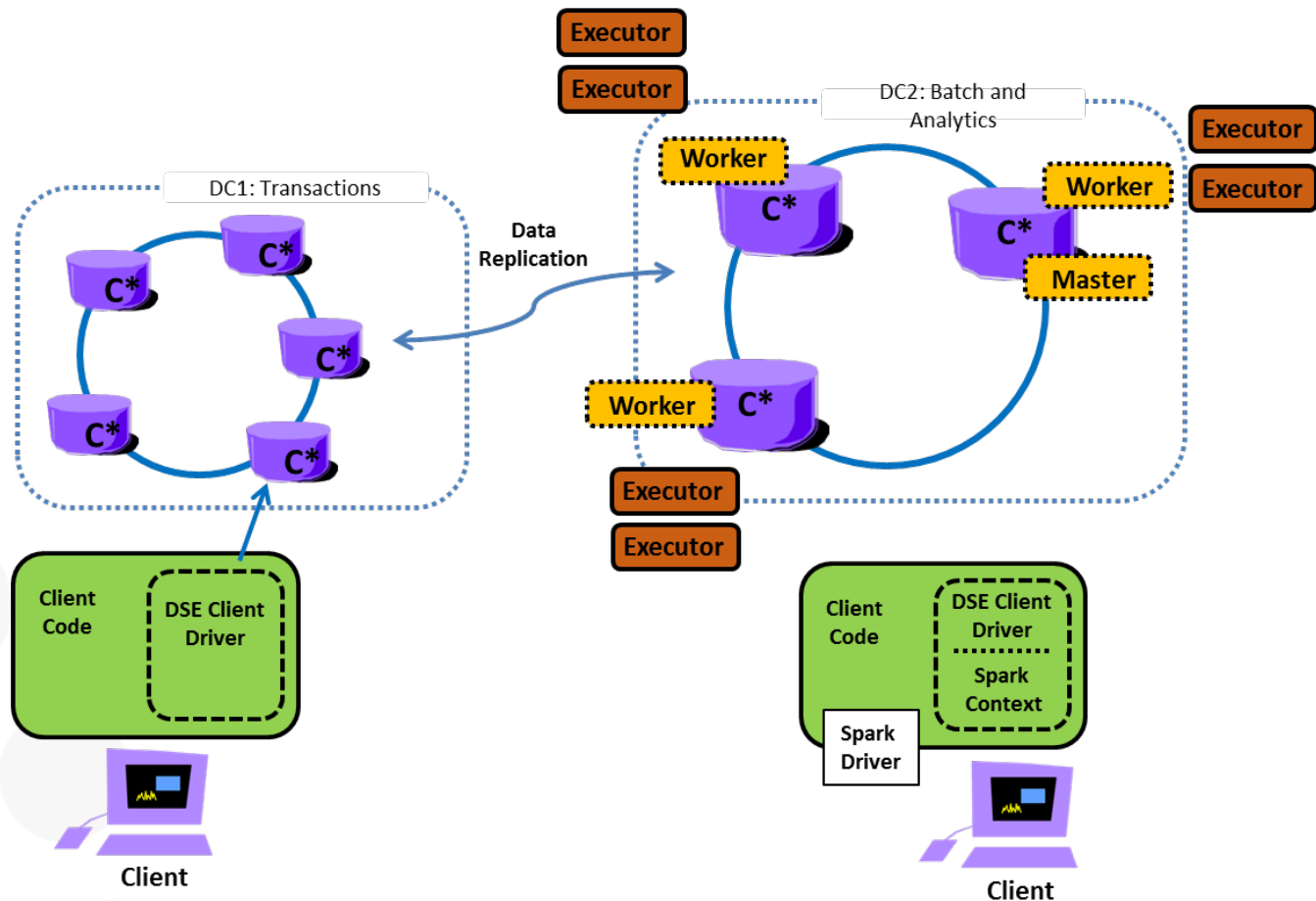
- Different number of Spark nodes than Cassandra nodes
 - In DSE, collocated Spark/Cassandra nodes means 1:1 sizing
 - DSE 5.1+ supports “Spark-only DCs” and “Spark-only Clusters”
- Separate Spark and Cassandra processing to avoid resource contention
 - While Spark can be somewhat controlled, Cassandra is less well behaved
 - The sacrifice is that reads are remote
- Newer version of Spark than is in DSE
 - Largely this is spurious. Most customers can't point to new feature they need (but some really can)
 - That said, Spark does not keep bug-fixing old releases
 - There has not been a Spark release past X.Y.2 (e.g., there was no 2.0.3)
 - Spark has performance optimizations to their core in new releases
 - The Catalyst optimizer has been a focus and will continue to be
- **Separate/existing customer team for Spark**

DSE Analytics (Apache Spark):

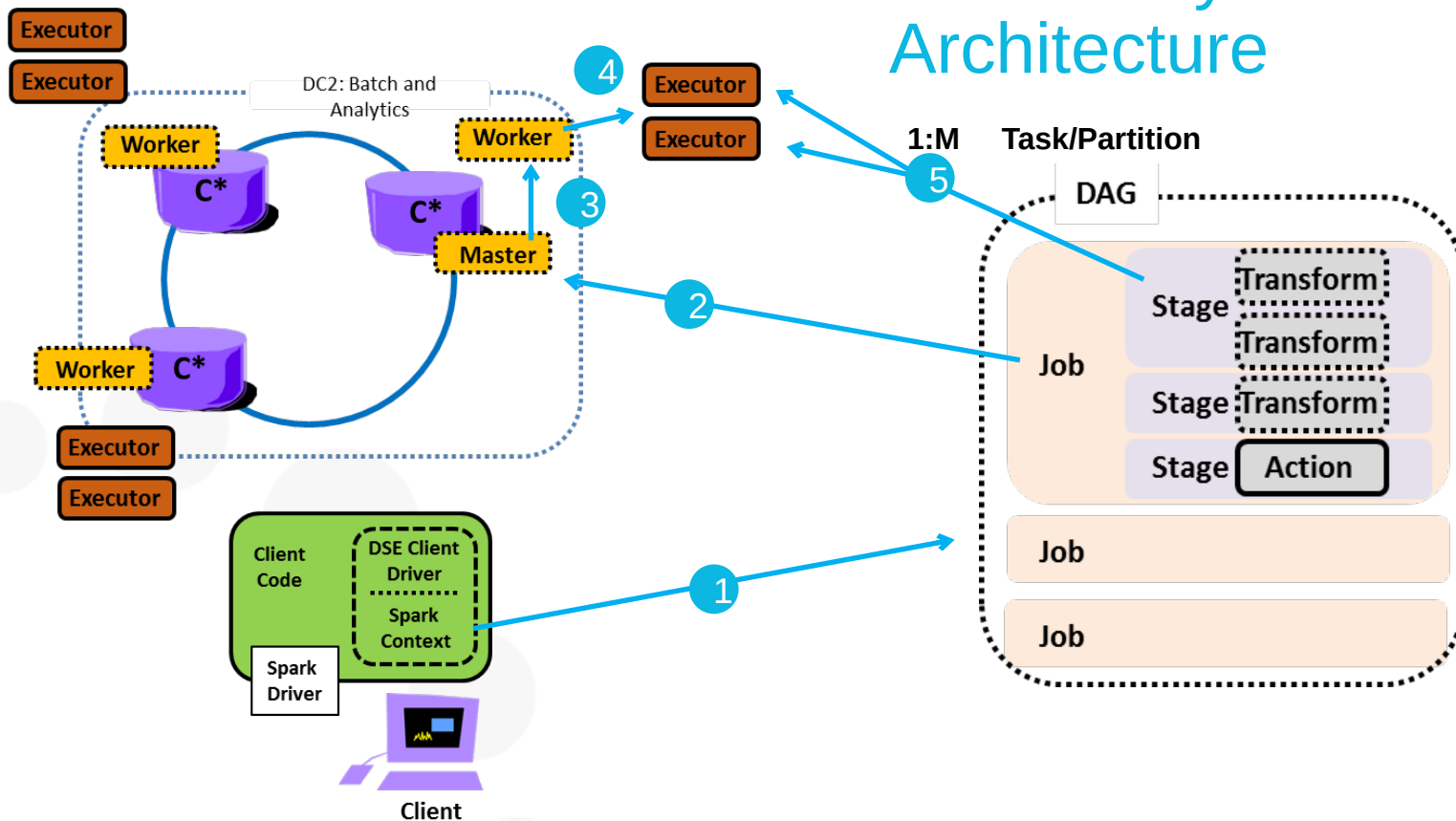
Operational Details

- DSE Analytics process architecture
- User Interfaces
- [The] Primary Spark API; RDDs, DataFrames, Dataset

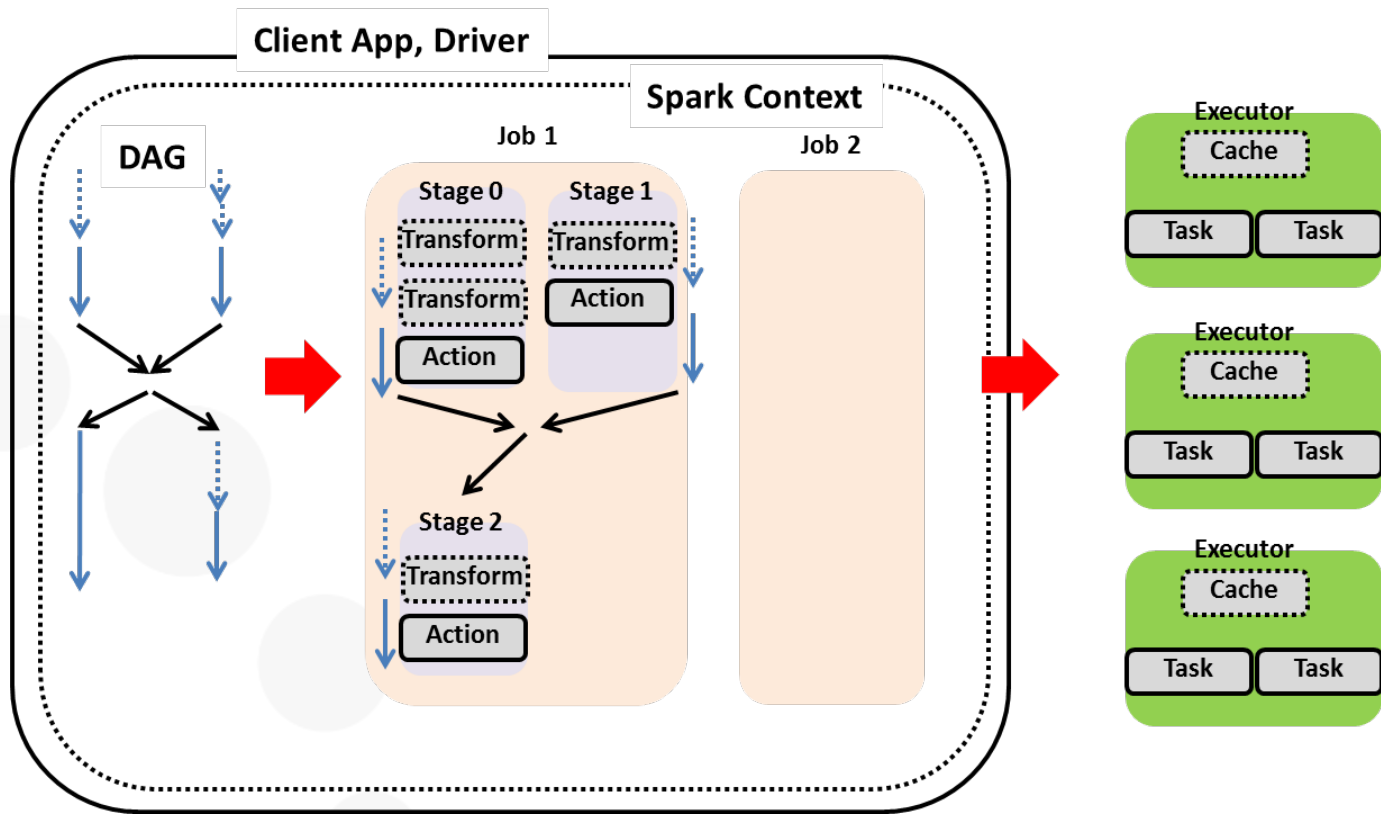
DSE Analytics: Process Architecture



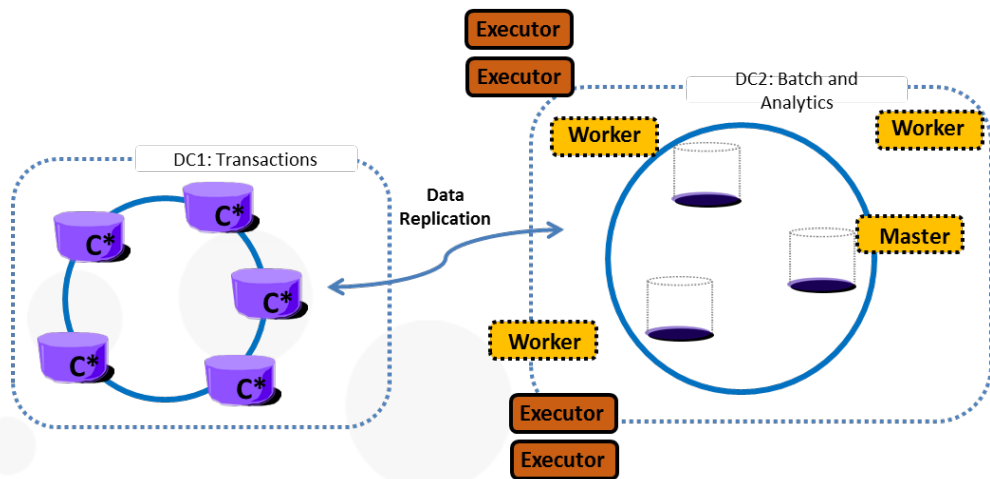
DSE Analytics: Process Architecture



DSE Analytics: Process Architecture

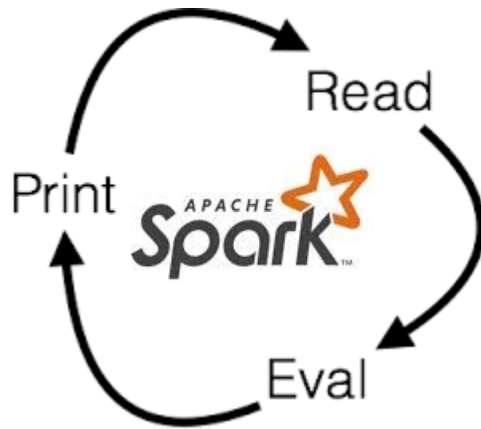


DSE Analytics: Process Architecture, (Outlier topics)



- DSE Analytics Streaming; one executor only, always on (?)
- Driver can be hosted locally or remote
- DSE 5.1+, can have analytics only nodes; no local data
- For executors, data must be available via same path on every node; DSEFS

- dse spark
:quit
- dse spark-sql
quit;
- dse pyspark
quit()
- dse sparkR
(Will fail without R)



DSE Analytics: UIs

- dse spark-submit xxx.jar



Java, Scala, Python, R
will automatically find Spark Master

- http:// (master node) :7080
- http:// (driver host) :4040 41, 42
- (Other)

DSE Analytics: Resource Math

Running Applications

Application ID		Name	Workpool	Cores	Memory per Executor	Submitted Time	User	State	Duration
app-20180709084020-0001	(kill)	Spark shell	default	0	1024.0 MB	2018/07/09 08:40:20	anonymous	WAITING	2 s
app-20180709083945-0000	(kill)	Spark shell	default	3	1024.0 MB	2018/07/09 08:39:45	anonymous	RUNNING	36 s

On the server; dse.yaml, spark-env.sh, other

- Tiered, many, non-trivial
- RAM, Cores, other
- See Url

On the client; in code, or ENV

- Subject to server limits
- RAM, Cores, other
- See Url

DSE Analytics: Programming Model

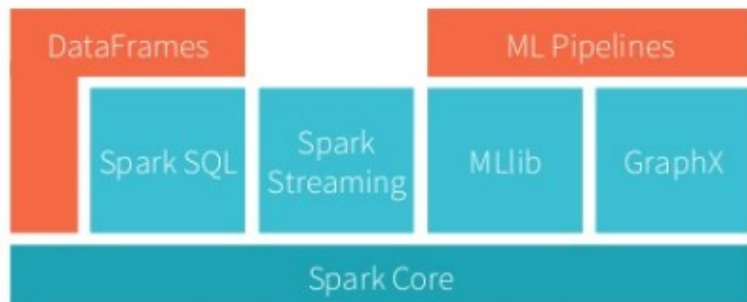
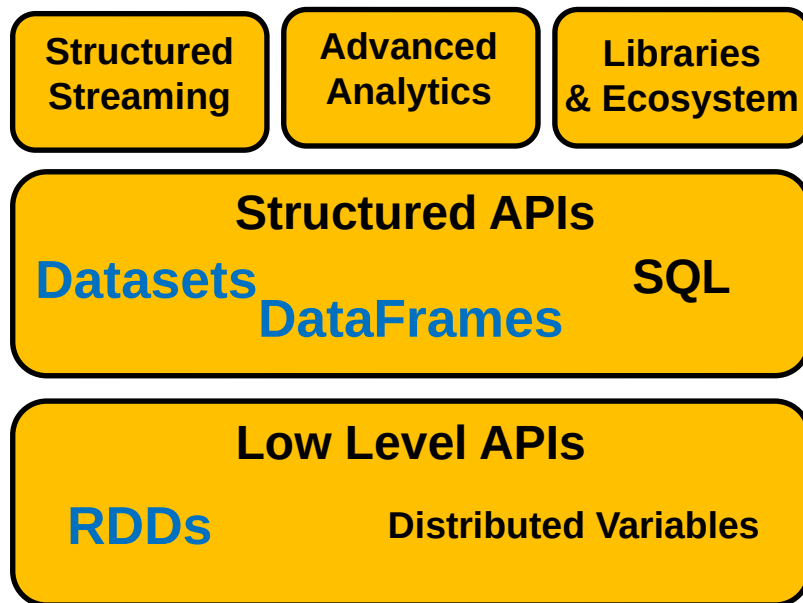
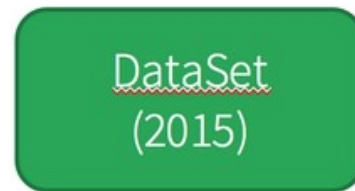
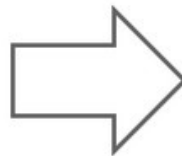
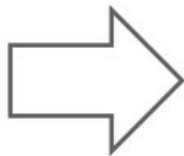


Image Source: DataBricks.com



Source: <https://www.amazon.com/Spark-Definitive-Guide-Processing-Simple/dp/1491912219>

DSE Analytics: Programming Model



Distribute collection
of JVM objects

Functional Operators (map,
filter, etc.)

Distribute collection
of Row objects

Expression-based operations
and UDFs

Logical plans and optimizer

Fast/efficient internal
representations

Internally rows, externally
JVM objects

Almost the “Best of both
worlds”: type safe + fast

But slower than DF
Not as good for interactive
analysis, especially Python

Image Source: DataBricks.com

DSE Analytics (Apache Spark):

Functional Programming

- What is Functional Programming (FP), and Why You Care

```
val names = List("Bob", "Nancy", "Dirk")
```

```
def myUC(Args: List[String]) : List[String] = {  
  var myList: List[String] = List()  
  //  
  for (s <- Args) {  
    myList = s.toUpperCase :: myList  
  }  
  myList  
}
```

```
val capNames = myUC(names)  
capNames.foreach(println)
```

Scala: What does
this fragment
output ?

Scala: What do these fragments output ?

```
val names = List("Bob", "Nancy", "Dirk")
```

```
val capNames = for (e <- names) yield e.toUpperCase  
capNames.foreach(println)
```

```
val capNames = names.map(_.toUpperCase)  
capNames.foreach(println)
```

```
val nums = List(1, 2, 3)  
val nums2 = nums.map(_ * 2)  
nums2.foreach(println)
```

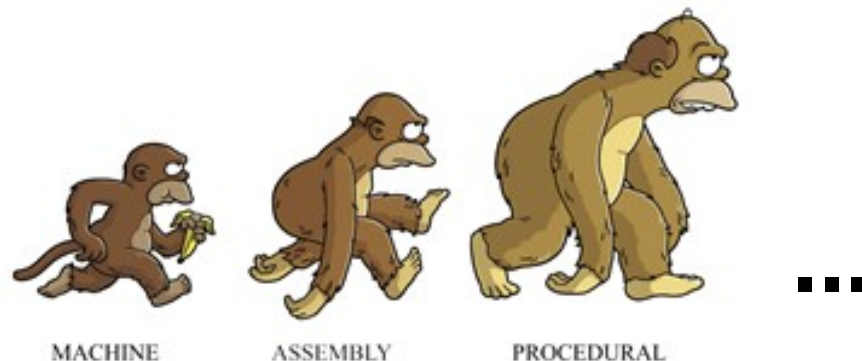
Definition of: map()



- Factor out common patterns
- Iterate over List elements
- Term comes from the mathematics domain
- Things that can be mapped over are called functors
- Methods of Scala's collections classes: `drop()`, `filter()`, `map()`, `reduce()`

Functional Programming (FP): Why Do You Care ?

- 98% of the examples you seek to use from books and on Google, will be FP (you need to be able to read)
- Knowing FP required to move from beginner to beyond beginner
- The best way to write **parallel and concurrent applications**; thread safe, no sync issues



Source: <https://medium.com/@cscalfani/so-you-want-to-be-a-functional-programmer-part-1-1f15e387e536>

Functional Programming (FP): Why Do You Care ?

FP Language Features-

- **Immutable data**, first class functions, tail call optimization

FP Programming Techniques-

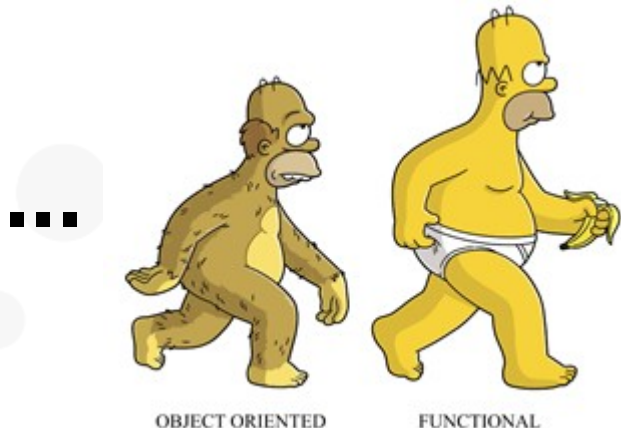
- **Mapping**, reducing, **pipelining**, **recurring**, currying, higher order functions (HOF)

FP Advantages-

- **Easier to parallelize**, **lazy evaluation**, **easier to test**, determinism



Which
entity in
the Spark
RT is
pipelining
?



Source: <https://medium.com/@cscalfani/so-you-want-to-be-a-functional-programmer-part-1-1f15e387e536>

$$\text{PF} == \text{ODI} + \text{NSE}$$

Pure Functions	Output Depends on Input	No Side Effects
-------------------	-------------------------------	--------------------

FP is the opposite of imperative programming

DSE Analytics: Intermediate and above level

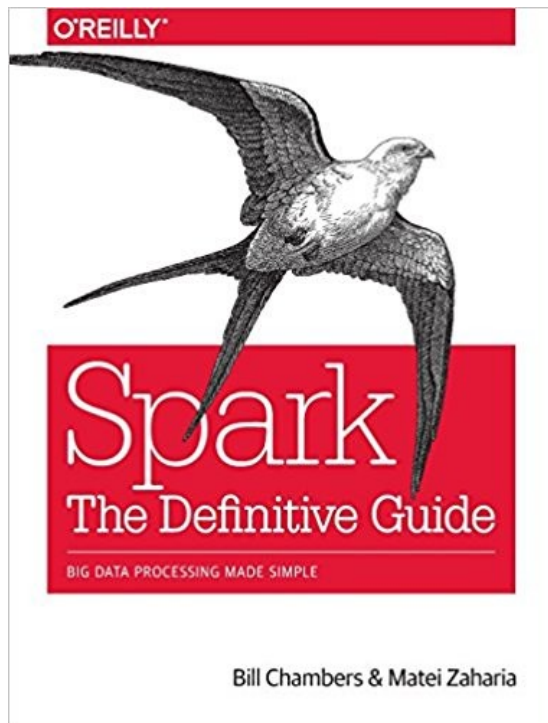
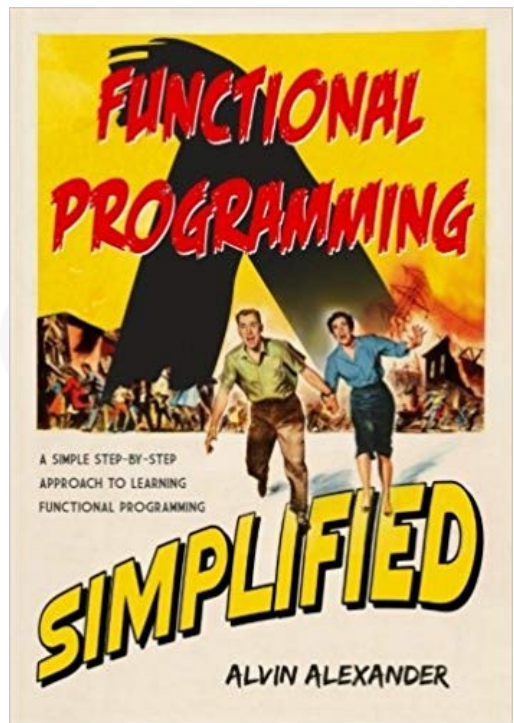


Functional
Programming



- FP, and imperative programming styles
- FP for transforms, imperative for UI
- What Spark is written in; examples, other
- Immutability, thread safe, concurrency, out of the box
- Rich Java package library

DSE Analytics: Must Reads ?





End of Unit:



Additional Detail:

History of Apache Spark

A Brief History of Spark-

- Spark was initially started by Matei Zaharia at UC Berkeley's AMPLab in 2009, and open sourced in 2010 under a BSD license.
- In 2013, the project was donated to the Apache Software Foundation and switched its license to Apache 2.0. In February 2014, Spark became a Top-Level Apache Project.
- In November 2014, Spark founder M. Zaharia's company Databricks set a new world record in large scale sorting using Spark.
- Spark had in excess of 1000 contributors in 2015, making it one of the most active projects in the Apache Software Foundation and one of the most active open source big data projects.

History of Apache Spark

Spark was originally developed to address the limitations of MapReduce.

- It is a distributed computing system, utilizing many machines to do a job
- While MapReduce reduces and writes to disk, Spark was developed as a shared memory system
 - Most work and data being held in memory
 - Spilled over to disk if memory capacity is met
- Because of this Spark is touted as being orders of magnitude faster than MapReduce
- Major speed enhancements came out it
 - 1.6 with solidifying DataFrames
 - 2.0 with solidifying DataSets and Tungsten execution backend
 - 2.2 Structured Streaming
- Easier to program and code
- Can do analytics in real time as well as batch
 - MapReduce is not a real time analytics engine

Spark is New, Fast Moving

Young and a very active open source project

- It has moved fast in the last couple of years
- Version changes coming very quickly
- Major API changes
- It is anticipated that like many open source projects this will slow down and the API will start becoming more stable
- But for now still very inventive
- This implies DSE may lag a few releases behind
 - 5.1.x DSE comes with Spark 2.0.x
 - 2.1.0 of Spark was already released when 5.1.0 was released
 - 6.0.0 released with the latest, Spark 2.2.x

Spark Cluster Managers:

Spark Standalone

The Spark Standalone cluster manager is a simple cluster manager available as part of the Spark distribution. It has HA for the master, is resilient to worker failures, has capabilities for managing resources per application, and can run alongside of an existing Hadoop deployment and access HDFS (Hadoop Distributed File System) data. The distribution includes scripts to make it easy to deploy either locally or in the cloud on Amazon EC2. It can run on Linux, Windows, or Mac OSX.

Apache Mesos

Apache Mesos, a distributed systems kernel, has HA for masters and slaves, can manage resources per application, and has support for Docker containers. It can run Spark jobs, Hadoop MapReduce, or any other service application. It has API's for Java, Python, and C++. It can run on Linux or Mac OSX.

Hadoop YARN

Hadoop YARN, a distributed computing framework for job scheduling and cluster resource management, has HA for masters and slaves, support for Docker containers in non-secure mode, Linux and Windows container executors in secure mode, and a pluggable scheduler. It can run on Linux and Windows.

Or, Datastax Analytics

Source: <http://www.agildata.com/apache-spark-cluster-managers-yarn-mesos-or-standalone/>

Useful Links:

- <https://spark.apache.org/>
- <http://spark.apache.org/docs/2.0.2/programming-guide.html>
- <http://spark.apache.org/docs/2.0.2/configuration.html>
- <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (not quite there for DSE 5.1)
- <https://github.com/datastax/spark-cassandra-connector/tree/master/doc>
- <https://www.youtube.com/watch?v=cKIHRD6kUOc&index=40&list=PLm-EP1kBI3YoiA-02vufoEj4CgYvIQglk> (Russ!)
- <https://support.datastax.com/hc/en-us/articles/204939759-Spark-shell-with-SSL-enabled-clusters>
- <http://www.bigsynapse.com/controlling-the-number-of-partitions-in-spark>
- <http://blog.cloudera.com/blog/2015/03/how-to-tune-your-apache-spark-jobs-part-1/>
- <https://www.dezyre.com/article/top-10-machine-learning-algorithms/202>
- <http://c2fo.io/c2fo/spark/aws/emr/2016/07/06/apache-spark-config-cheatsheet/>
- <http://www.sparktutorials.net/analyzing-flight-data:-a-gentle-introduction-to-graphx-in-spark>

Useful Links:

- <https://databricks.com/blog/2016/10/03/voice-from-cern-apache-spark-2-0-performance-improvements-investigated-with-flame-graphs.html>
- <https://databricks.com/blog/2016/05/23/apache-spark-as-a-compiler-joining-a-billion-rows-per-second-on-a-laptop.html>
- <https://www.gitbook.com/book/jaceklaskowski/mastering-apache-spark/details>
- <https://www.analyticsvidhya.com/blog/2015/07/difference-machine-learning-statistical-modeling/>
- <http://www.doanduyhai.com/blog/?p=2325> (Setting up Zeppelin)
- <https://databricks.com/blog/2017/10/19/introducing-natural-language-processing-library-apache-spark.html>
- <https://databricks.com/blog/2017/08/31/cost-based-optimizer-in-apache-spark-2-2.html>
- <http://www.stratio.com/blog/optimizing-spark-streaming-applications-apache-kafka/>
- <https://techvidvan.com/tutorials/apache-spark-performance-tuning/>
- <http://shop.oreilly.com/product/0636920046967.do> (High Performance Spark Book)
- <https://www.gitbook.com/book/jaceklaskowski/mastering-apache-spark/details> (Book)